VYUHA · VR EDITION · 18 OCTOBER 2026

Your training ground, seen from inside.

A guide to the new VR observation post, written for someone who has never used virtual reality.

> **You can try it today without a headset.**
> Open Training ground → Open VR cockpit. The desktop preview uses the same cockpit, exercise and decision controls.

> **What “tested” means here**
> The software has automated tests and browser tests using an emulated headset. No physical headset was available. Real headset comfort, tracking, controller feel and frame rate remain untested. This is an implemented VR prototype with software evidence, not hardware certification.

Read in this order

What VR adds

Try it on your laptop

Read the cockpit

Make a decision

Use a real headset later

Pause, exit and troubleshoot

Understand the evidence

How it works / maintain it

The ordinary VYUHA workspace remains available. This guide describes the new working tree, not a new tagged software release.

What are we actually building?

Virtual reality (VR) means a headset shows a computer-made world in front of your eyes. Each eye gets its own view. As you turn your head, the view changes. Controllers let you point at things and select them.

VYUHA's VR version places you at a fixed observation post above the existing training terrain. A control console sits in front of you. You look at the simulated aerial contacts, read the available sensor evidence, and record your decisions.

This uses **WebXR**: the browser's connection to a compatible headset. There is no separate headset app to install. The headset browser opens VYUHA, asks for permission, and starts the immersive view.

Ordinary VYUHA	VR observation post
A screen, mouse and ordinary web controls.	A stereo headset view, head tracking and controller rays.
The observation camera can focus on a selected contact.	Your head controls where you look. Selecting a contact highlights it without turning your head for you.
Radar and other evidence appear in panels.	Detected contacts and available radar, IR, RF and IFF evidence appear on console surfaces inside VR.
Exercises run in the browser simulation worker.	The same worker runs the exercise and accepts the same action messages.
The server reconstructs decisions and verifies scores.	The same server verification and after-action review apply.

Why sit at a fixed post?

The app does not push you around, fly the camera or use joystick movement. This gives the experience a stable reference and a simple control scheme. It is intended for seated use. That design choice still needs a comfort trial with a real headset.

> The terrain, aircraft shapes, sensor behaviour and response rules are illustrative training models. VR makes the presentation immersive; it does not make these models operationally accurate.

02 / TRY IT NOW

Your first laptop preview

1. Start VYUHA as usual. From the project folder, run `bash scripts/start-local.sh`. Initial setup may need Internet. If a server is already running, use that server or stop it before reusing its port.
2. Open `http://127.0.0.1:8000` in your browser. Choose **Training ground**.
3. Find the **VR observation post** card below the graphics settings. Select **Open VR cockpit**.
4. The message will normally say no immersive headset was detected. This is expected. **Enter VR** stays disabled; the preview and its controls still work.
5. Select **Begin exercise** inside the cockpit or under **Desktop preview controls**.
6. Select a contact such as **T2**. Follow the designation, classification and response steps on the next pages.
7. Use **Pause exercise** whenever you need time. After 75 seconds of running time, the ordinary review opens. Wait for **Server verified**.

Two ways to click

You can click the drawn console buttons, or use the normal buttons below the preview. These buttons are also usable by keyboard. Both send the same actions.

Choose the exercise before entering a headset

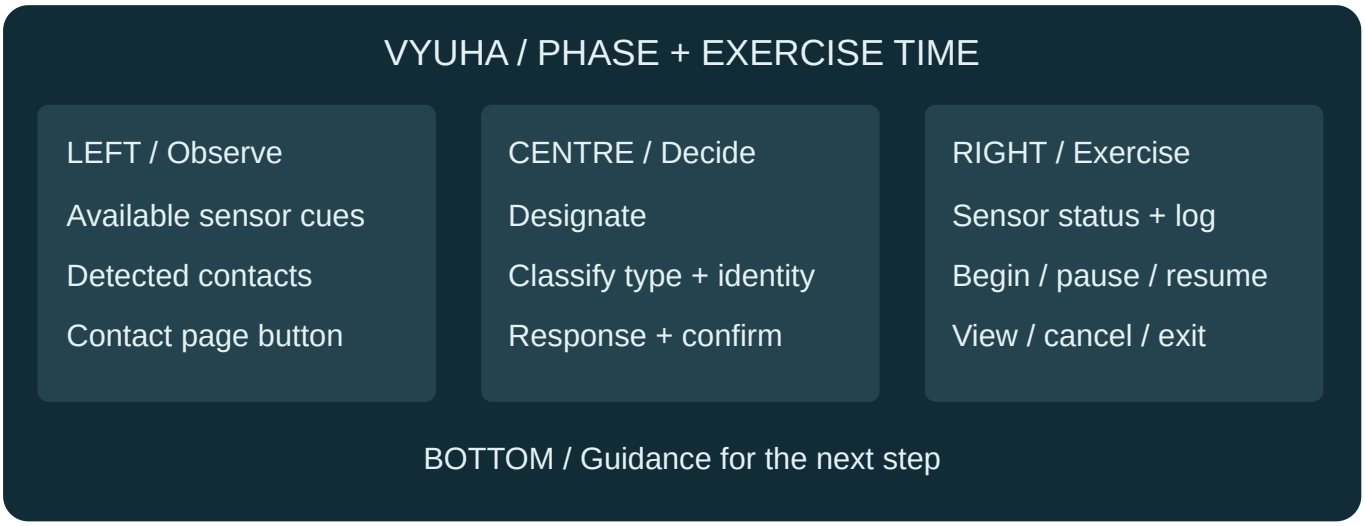
The normal setup controls still select the trainee, day/night conditions, saved exercise or adaptive drill. An adaptive drill may first show **Prepare adaptive drill**, then **Begin exercise**. Preparation requires the local server. Assigned exercises can be accepted using the existing workspace controls.

What if the server is off?

The original scripted exercise can fall back to device-only practice. Decisions can be saved in browser storage; the review says verification is pending. Advanced/adaptive and library exercises require their original local server. “Saved” and “server verified” are different states.

> Desktop preview checks the software and lets you learn the workflow. It cannot show you what headset lenses, controller weight or motion comfort feel like.

Three panels, one training picture



Term	What it means
Contact / T1, T2…	A simulated object with a tracking label. A label does not tell you whether it is hostile.
Radar	A modelled position-detection sensor. NORMAL, DEGRADED and OFFLINE describe its current availability. The selected contact can still have no radar return at that moment.
IR / thermal	The modelled infrared sensor and illustrative heat-style view. This is not a real thermal camera.
RF	Radio-frequency evidence. “Link detected” is a cue about a radio link. “No link detected” is not proof of identity.
IFF	Identification friend or foe. A “friendly reply” is a simulated cue; “no reply” does not automatically mean hostile.
Declared	The type and identity you recorded. Unknown means you have not classified it.

Four contact buttons fit on a page inside VR. The contact-page button cycles through more. Contacts appear in the list after a working sensor has detected them. Sensor outages are shown as **OFFLINE**, rather than a made-up negative reading.

Observe → designate → classify → respond

1. **Select a detected contact.** Point at its label and release the trigger, or click it on your laptop. The left panel shows the evidence for that contact.
2. **Designate.** This records that you chose to examine the contact. Classification stays disabled until designation.
3. **Choose its type.** The Type button cycles through Quadcopter, Fixed-wing, No-RF FPV and Bird. The choice is your answer; the app does not fill in the correct answer.
4. **Choose its identity.** The Identity button cycles through Hostile, Friendly and Neutral. Select **Confirm classification** to submit both choices.
5. **Choose a response.** Select Hold fire, RF jammer or Kinetic response, where the exercise permits it. This first selection arms a pending response; it does not execute it.
6. **Check the target and confirm.** The confirmation button names the response and contact. Select it to send the decision. Select Cancel to clear it. A pending response expires after eight seconds and clears when the selected contact or exercise phase changes.
7. **Read the exercise log.** It reports what happened under the model's rules. Continue observing and choosing contacts.

Response	Meaning inside this prototype
Hold fire	Record a decision to keep monitoring. It does not pause time or remove the contact.
RF jammer	A modelled radio-link response. It can fail for a no-RF contact or outside the permitted range.
Kinetic response	A modelled physical response. Range and restricted-zone rules still apply.

Example: Suppose the cues lead you to classify T2 as a friendly quadcopter. You designate it, select Quadcopter + Friendly, confirm the classification, select Hold fire, then confirm Hold on T2. That is an example of the controls, not a supplied answer for every scenario.

> A button being enabled means you may submit that decision. It does not mean the decision is correct or effective. The shared simulation applies the rules and the server later verifies the recorded result.

Connect it to the same project

You need a headset/browser combination that supports **immersive WebXR**, controller selection, and WebGL. Meta Quest browser with tracked controllers is the software emulator profile used here; this is not a claim of a physical Quest test or of support for every headset. Hand-only control and phone cardboard viewers are outside this version's tested scope.

The important difference: HTTPS

Browsers protect access to headset hardware. A laptop's `http://127.0.0.1` address can be treated as trustworthy locally. On a headset, `127.0.0.1` means the headset itself. To reach your laptop, use its LAN address over **HTTPS with a certificate trusted by the headset**.

A certificate is the website's digital identity. A self-signed certificate or clicking past a warning may not make WebXR available. The certificate must cover the exact hostname or IP address you open, and its issuing authority must be trusted on the headset. Have the person managing your network set that up if it is unfamiliar.

Launch with the existing HTTPS support

After normal project setup, put your certificate and private key paths into these variables. These are placeholders; substitute your actual files.

```
export VYUHA_SSL_CERTFILE=/absolute/path/server-cert.pem
export VYUHA_SSL_KEYFILE=/absolute/path/server-key.pem
bash scripts/start-vr.sh --build
```

1. Put laptop and headset on the same trusted network. Allow the chosen server port through the laptop's firewall.
2. Open the printed `https://…:8000` LAN URL in the headset browser. The page should have no certificate warning.
3. Join the unit using a callsign if prompted. Open Training ground → Open VR cockpit.
4. Sit with clear space for your hands. Use the headset's normal boundary/recentre controls to face forward.
5. Select **Enter VR** and allow the browser's permission prompt. Point either controller at a console button and release its trigger to select.

The launcher enforces certificate configuration and uses existing Unit Mode. It does not generate or install certificates. Keep this prototype on a trusted network; it has no production authentication.

Pause, leave, and recover

Pause exercise stops simulation time. **Resume exercise** continues it. You can do both inside VR. The headset's system menu can also end the immersive session.

Leaving VR pauses an active drill. When WebXR reports hidden or blurred visibility, the app also pauses. Returning to visibility does not automatically resume. When the exercise ends normally, VR closes and the ordinary after-action review opens. Review, exercise authoring and unit administration remain ordinary web pages.

What you see	What to do
Enter VR is disabled on a laptop	Expected without a compatible immersive headset. Use Desktop preview controls.
"Headset mode needs trusted HTTPS"	Open a trusted HTTPS address on the headset. Plain LAN HTTP supports preview only.
Permission denied / VR could not start	Allow the site's headset permission, check the headset connection and retry Enter VR. Preview remains usable.
Buttons look dim	Start or resume; select an active detected contact; designate it; confirm its classification. Some exercises restrict available responses.
Confirmation disappeared	It timed out after eight seconds, you changed contact or phase, or you cancelled it. Choose a response again.
RF or radar is OFFLINE	This can be an exercise condition or instructor command. Use the evidence that is actually available.
Black screen / graphics context lost	The loss handler pauses and ends VR. Close and reopen the cockpit; try the ordinary tactical view if rendering still fails.
Verification pending	Check the original local server connection and use Retry verification in the review. Keep the locally saved session or export it.

Your first real trial should be short

Check text clarity, controller aiming and the pause/exit buttons before running a full drill. Stop if you feel uncomfortable. Keep walking and joystick movement out of the trial. Record the headset, browser version, comfort observations and performance instead of assuming the emulator proves them.

Evidence, with clear boundaries

The validation report at `docs/vr/VALIDATION.md` and the raw browser results at `docs/vr/evidence/browser-results.json` record the checks for this version.

Check	What it establishes
Frontend and backend regressions	The automated suites pass, including existing simulation parity checks. This protects the shared training behaviour.
VR action tests	Decisions obey selection/phase rules, preserve available evidence, and replay through the ordinary scoring engine.
XR lifecycle tests	Denial, failed setup, interrupted entry, visibility loss, exit and re-entry have defined cleanup behaviour.
Desktop browser test	The cockpit renders, its controls work, and submitted decisions reach a server-verified review.
IWER emulated headset test	A WebXR session renders through the real Three.js integration; two simulated controller rays operate scene buttons. A full-duration drill completes and is verified.
Physical headset / comfort / device FPS	**Not tested.** Real lenses, tracking, display timing, controller ergonomics and sustained device performance require hardware.

The test browser uses software graphics on this laptop. Its speed is not a Quest performance measurement. Existing desktop/Android FPS figures from earlier releases also do not establish VR frame rate.

Before calling it ready for a particular headset

1. Record device, browser, server and certificate setup.
2. Confirm correct eye height, head tracking, readable text and both controller rays.
3. Try designation, classification, response confirmation, timeout, pause, headset removal/system menu and re-entry.
4. Finish several drills, including a sensor outage and an adaptive/library exercise. Check server verification and replay.
5. Measure sustained headset frame rate and session behaviour under the intended workload. Record comfort feedback and any dropped tracking.

> Passing a software emulator is useful evidence. It cannot honestly prove that every headset is comfortable, fast, compatible or ready for deployment.

One simulation, two presentations

```
Ordinary workspace OR VR console
               ↓ same action payloads
Browser worker: simulation + accepted decisions
               ↓ ordered event log
Local server: reconstruct and verify score
               ↓
Existing after-action review / replay / training profile
```

`VRCockpit.tsx` draws the console and reuses the shared world. Console labels are textures on real 3D surfaces, so headset controls do not depend on a browser HTML overlay. Both controllers use rays to intersect those surfaces.

`vr.ts` contains action gating, observed evidence and session lifecycle handling. The browser must report `immersive-vr` support. A user click requests a `local` reference space; a fixed 1.6 m rig offset puts the seated origin at the console. No camera-follow routine runs during VR.

The worker stays at 30 Hz. WebXR renders independently at the device's display cadence. VR uses the low-detail world, pixel ratio 1, a requested 0.85 framebuffer scale and maximum foveation where supported. These are conservative rendering settings, not measured device-performance guarantees. The ordinary observation renderer is removed while the VR preview is open.

Repeat the checks

```
# From the project folder:
.venv/bin/python -m pytest server/tests -q
cd client
npm ci
npm test
npm run build
cd ..
.venv/bin/python scripts/test-vr-browser.py
```

The browser test needs the Python Playwright package and its Chromium browser installed in your test environment. It starts its own temporary server and database. IWER 2.6.0 is a pinned development dependency, injected only by the test. It is not shipped as a headset substitute in the production application.

Primary technical references

Three.js WebXRManager
MDN: WebXR permissions and security
MDN: starting and stopping sessions
Meta: Immersive Web Emulation Runtime